

Formation RAG

Retrieval-Augmented Generation

Séance 1 - Fondements théoriques

David Christ MEKONTCHOU Nzone

Mai 2026

Document de cours

Ce document accompagne la première séance de la formation *Retrieval-Augmented Generation (RAG)*. Il est destiné à servir de référence aux apprenants. Il couvre les fondements théoriques nécessaires avant la mise en pratique des séances 2 et 3.

Public visé : Développeurs, data scientists, étudiants ou professionnels souhaitant découvrir les systèmes RAG.

Prérequis : Notions de programmation, curiosité pour l'IA. Aucune connaissance préalable en NLP requise.

Table des matières

1	Introduction et objectifs	5
1.1	Pourquoi cette formation ?	5
1.2	Objectifs pédagogiques de la formation complète	5
1.3	Plan de la séance 1 (théorie)	5
2	Fondations : IA, Machine Learning, Deep Learning et IA générative	6
2.1	De l'intelligence artificielle au Deep Learning	6
2.2	Le Machine Learning en une page	6
2.3	Deep Learning et réseaux de neurones	7
2.4	IA générative	7
3	Les Large Language Models (LLM)	7
3.1	Définition et intuition	8
3.2	Bref historique	8
3.3	Notions clés vulgarisées	8
3.3.1	Token	8
3.3.2	Embedding	9
3.3.3	Fenêtre de contexte	9
3.3.4	L'architecture Transformer	9
3.4	Que savent vraiment faire les LLM ?	10
4	Les limites des LLM : pourquoi le RAG ?	10
4.1	Limite n°1 : Les hallucinations	10
4.2	Limite n°2 : Le knowledge cutoff	11
4.3	Limite n°3 : Pas d'accès aux données privées	11
4.4	Limite n°4 : Absence de sources et de traçabilité	11
4.5	Limite n°5 : La fenêtre de contexte	11
4.6	Limite n°6 : Le coût du fine-tuning	11
4.7	Synthèse des limites	12
5	Positionnement du RAG : Prompt Engineering vs Fine-tuning vs RAG	12
5.1	Prompt Engineering	12
5.2	Fine-tuning	13
5.3	RAG	13
5.4	Matrice de décision	13
6	Qu'est-ce qu'un système RAG ?	13
6.1	Définition complète	14
6.2	L'analogie pédagogique : l'examen à livre ouvert	14
6.3	Origine historique	14
6.4	Schéma général	14
6.5	Ce qu'un RAG n'est pas	14

7	Fonctionnement détaillé d'un RAG	15
7.1	Phase 1 — L'indexation (offline)	15
7.1.1	Étape 1.1 — Chargement des documents	15
7.1.2	Étape 1.2 — Le chunking (découpage)	15
7.1.3	Étape 1.3 — La vectorisation (embeddings)	16
7.1.4	Étape 1.4 — Stockage dans un vector store	16
7.2	Phase 2 — Retrieval + Generation (online)	16
7.2.1	Étape 2.1 — Vectorisation de la requête	17
7.2.2	Étape 2.2 — Recherche de similarité	17
7.2.3	Étape 2.3 — Construction du prompt augmenté	17
7.2.4	Étape 2.4 — Génération par le LLM	17
8	Les composants techniques d'un RAG	18
8.1	Document loaders	18
8.2	Text splitters / chunkers	18
8.3	Modèles d'embeddings	18
8.4	Vector stores	19
8.5	LLM de génération	19
8.6	Frameworks d'orchestration	19
9	Les types de RAG	19
9.1	Naive RAG (RAG de base)	19
9.2	Advanced RAG	20
9.2.1	Pré-retrieval	20
9.2.2	Post-retrieval	20
9.3	Modular RAG	20
9.4	Agentic RAG	20
9.5	Self-RAG	21
9.6	Corrective RAG (CRAG)	21
9.7	GraphRAG	21
9.8	HyDE	21
9.9	Multi-modal RAG	21
9.10	Tableau comparatif	22
10	Évaluation des systèmes RAG	22
10.1	Pourquoi évaluer ?	22
10.2	La notion de golden dataset	23
10.3	Métriques de retrieval	23
10.4	Métriques de génération	23
10.5	Frameworks d'évaluation	24
10.6	La boucle d'amélioration	24
11	Avantages, limites et pièges du RAG	24
11.1	Les avantages	24

11.2 Les limites	25
11.3 Pièges classiques	25
12 Cas d’usage concrets	25
12.1 Assistants documentaires d’entreprise	25
12.2 Support client	25
12.3 Assistants juridiques	26
12.4 Assistants médicaux	26
12.5 Assistants techniques / DevOps	26
12.6 Assistants éducatifs	26
12.7 Recherche académique et veille	26
13 Écosystème et outils	26
13.1 LangChain	26
13.2 Alternatives	27
13.3 Providers LLM	27
13.4 Providers embeddings	27
14 Préparation aux séances pratiques	27
14.1 Ce que nous allons construire	27
14.2 Prérequis techniques	28
14.3 Pour aller plus loin avant la séance 2	28
Glossaire	28
Bibliographie sélective	29

1 Introduction et objectifs

1.1 Pourquoi cette formation ?

L'intelligence artificielle générative, et en particulier les *Large Language Models* (LLM) comme GPT-4, Claude ou Llama, ont transformé en profondeur notre rapport à l'information depuis 2022. Pourtant, malgré leur puissance, ces modèles présentent des limites fondamentales qui rendent leur utilisation en entreprise risquée : hallucinations, méconnaissance des données privées, impossibilité d'accéder à des informations récentes, absence de citations.

Les systèmes **RAG** (*Retrieval-Augmented Generation*) sont apparus comme la réponse pragmatique à ces limites. Ils permettent de combiner la fluidité d'un LLM avec la précision d'une base documentaire métier. C'est aujourd'hui l'une des architectures les plus déployées en production pour les assistants IA d'entreprise.

1.2 Objectifs pédagogiques de la formation complète

À l'issue des trois séances, vous serez capable de :

- comprendre le fonctionnement interne d'un système RAG ;
- identifier les cas d'usage où un RAG est pertinent ;
- distinguer les différents types de RAG et choisir l'architecture adaptée ;
- évaluer la qualité d'un système RAG avec des métriques pertinentes ;
- implémenter un RAG complet avec LangChain ;
- identifier les pièges classiques et appliquer les bonnes pratiques.

1.3 Plan de la séance 1 (théorie)

1. Fondations : IA, ML, Deep Learning et IA générative
2. Les LLM : capacités et fonctionnement vulgarisé
3. Les limites des LLM : pourquoi le RAG est-il nécessaire ?
4. Positionnement du RAG vs Fine-tuning vs Prompt Engineering
5. Qu'est-ce qu'un RAG ?
6. Fonctionnement détaillé : indexation et retrieval
7. Les composants techniques d'un RAG
8. Les types de RAG : du Naive au GraphRAG

9. L'évaluation des systèmes RAG
10. Avantages, limites et pièges
11. Cas d'usage concrets
12. Écosystème et outils
13. Préparation aux séances pratiques

2 Fondations : IA, Machine Learning, Deep Learning et IA générative

Avant de plonger dans les LLM puis les RAG, posons rapidement le vocabulaire. Ces notions sont souvent confondues alors qu'elles désignent des choses différentes mais imbriquées.

2.1 De l'intelligence artificielle au Deep Learning

Définition

Intelligence artificielle (IA) : discipline informatique visant à concevoir des systèmes capables d'effectuer des tâches qui requièrent normalement de l'intelligence humaine (raisonner, percevoir, apprendre, décider).

L'IA est un champ large qui inclut :

- l'**IA symbolique** (systèmes experts à base de règles) ;
- le **Machine Learning** (apprentissage à partir de données) ;
- le **Deep Learning** (sous-ensemble du ML basé sur des réseaux de neurones profonds).

[IMAGE À INSÉRER : diagramme en poupées russes $IA \supset ML \supset Deep Learning \supset Generative AI \supset LLM$ — facilement trouvable en cherchant « AI ML DL GenAI venn diagram » sur Google Images]

2.2 Le Machine Learning en une page

Le ML repose sur une idée simple : au lieu de programmer des règles à la main, on *apprend* ces règles à partir d'exemples.

Exemple

Pour détecter des emails spams, on ne code pas une liste de mots interdits. On donne au modèle des milliers d'exemples annotés (spam/non spam), et il découvre seul les corrélations utiles.

Trois grandes familles :

- **Apprentissage supervisé** : on connaît la réponse attendue (classification, régression).
- **Apprentissage non supervisé** : on cherche des structures sans étiquettes (clustering, réduction de dimension).
- **Apprentissage par renforcement** : un agent apprend par essais/erreurs en recevant des récompenses (RLHF, jeux, robotique).

2.3 Deep Learning et réseaux de neurones

Le Deep Learning utilise des **réseaux de neurones artificiels**, structures inspirées (très lointainement) du cerveau, organisées en couches. La « profondeur » fait référence au nombre élevé de couches.

À retenir

Le Deep Learning a explosé à partir de 2012 grâce à trois facteurs convergents : la disponibilité de **données massives**, la **puissance de calcul** (GPU) et des **algorithmes** plus matures (backpropagation, ReLU, etc.).

2.4 IA générative

Définition

IA générative : classe de modèles capables de *produire* de nouveaux contenus (texte, image, son, vidéo, code) qui n'existaient pas dans les données d'entraînement, mais qui en respectent les patterns.

Quelques modèles emblématiques :

- **Texte** : GPT-4, Claude, Llama, Mistral, Gemini.
- **Image** : DALL-E, Midjourney, Stable Diffusion.
- **Audio/Voix** : ElevenLabs, Whisper, MusicGen.
- **Vidéo** : Sora, Runway, Veo.
- **Code** : GitHub Copilot, Claude Code, Cursor.

3 Les Large Language Models (LLM)

3.1 Définition et intuition

Définition

Large Language Model (LLM) : modèle de Deep Learning entraîné sur d'immenses corpus de texte (souvent plusieurs milliers de milliards de mots), capable de comprendre et générer du langage naturel en prédisant le mot suivant le plus probable.

L'intuition centrale est étonnamment simple : un LLM est une **machine à compléter du texte**. Donnez-lui un début de phrase, il en produit la suite la plus plausible compte tenu de ce qu'il a vu pendant son entraînement.

Exemple

Entrée : « Le chat est sur le »

Sortie probable : « tapis », « canapé », « toit »...

Le modèle apprend à attribuer une probabilité à chaque mot du vocabulaire pour la position suivante.

3.2 Bref historique

Année	Étape
2013	Word2Vec (Google) — premiers embeddings de mots dans un espace vectoriel.
2017	Transformer (papier « Attention Is All You Need », Google) — architecture qui révolutionnera tout.
2018	BERT (Google) et GPT-1 (OpenAI) — premières grosses applications du Transformer.
2020	GPT-3 — 175 milliards de paramètres, démonstration du <i>few-shot learning</i> .
2022	ChatGPT — démocratisation grand public.
2023+	GPT-4, Claude 3, Gemini, Llama 2/3, Mistral, modèles open-weights.
2025+	Modèles raisonneurs (o-series, Claude avec extended thinking), multi-modalité avancée, contextes de 1M tokens.

TABLE 1 – Quelques jalons clés de l'évolution des LLM.

3.3 Notions clés vulgarisées

3.3.1 Token

Un **token** n'est pas exactement un mot. C'est l'unité de base manipulée par le modèle. Pour l'anglais, 1 token $\approx$ 4 caractères ou 0,75 mot. Pour le français, c'est généralement plus coûteux.

Exemple

La phrase « Bonjour, comment vas-tu ? » peut être tokenisée en : ["Bon", "jour", ",", " ", "comment", " ", "vas", "-", "tu", "?"] (≈ 8 tokens).

3.3.2 Embedding

Définition

Embedding : représentation vectorielle (une liste de nombres réels, par exemple 1536 dimensions) d'un mot, d'une phrase ou d'un document. Des contenus sémantiquement proches ont des vecteurs proches dans cet espace.

[IMAGE À INSÉRER : projection 2D d'embeddings montrant que « roi », « reine », « prince » sont proches, alors que « banane » est ailleurs — chercher « word embeddings visualization » sur Google Images]

Cette idée est **absolument centrale** pour comprendre le RAG : nous reviendrons en détail sur les embeddings dans la section dédiée à l'indexation.

3.3.3 Fenêtre de contexte

Définition

Fenêtre de contexte (context window) : nombre maximal de tokens qu'un LLM peut « voir » en entrée simultanément (prompt + sortie en cours).

Ordres de grandeur en 2026 :

- GPT-3.5 : 4 000 à 16 000 tokens
- GPT-4 Turbo : 128 000 tokens
- Claude 4 : 200 000 à 1 000 000 tokens
- Gemini : jusqu'à 2 000 000 tokens

3.3.4 L'architecture Transformer

Sans rentrer dans les mathématiques, retenir deux mécanismes :

- **Attention** : à chaque étape, le modèle décide quels tokens du contexte sont les plus pertinents pour produire le prochain. C'est ce qui lui permet de capturer des dépendances longues (« Le chat qui a mangé la souris est *noir* »).
- **Empilement de couches** : chaque couche affine la représentation interne. Les modèles modernes comptent des dizaines voire des centaines de couches.

3.4 Que savent vraiment faire les LLM ?

- Génération et résumé de texte
- Traduction
- Classification et extraction d'information
- Question/Réponse
- Génération et explication de code
- Raisonnement (de mieux en mieux)
- Dialogue et assistance conversationnelle

Questions de réflexion

1. Quelle est la différence entre IA, Machine Learning et Deep Learning ?
2. Pourquoi dit-on qu'un LLM est « juste » une machine à compléter du texte ?
3. À quoi sert un embedding ? Citez une intuition simple.

4 Les limites des LLM : pourquoi le RAG ?

Les LLM sont impressionnants, mais ils souffrent de **limites structurelles** qui rendent leur déploiement en entreprise délicat. Comprendre ces limites est la clé pour comprendre pourquoi le RAG existe.

4.1 Limite n°1 : Les hallucinations

Définition

Hallucination : phénomène par lequel un LLM produit une information *plausible mais factuellement incorrecte*, avec une assurance trompeuse.

Le LLM ne « ment » pas volontairement : il génère ce qui ressemble statistiquement à une bonne réponse. Quand il ne sait pas, il invente.

Exemple

Question : « Qui a remporté le prix Nobel de physique 2025 ? »

Réponse hallucinée : « Le prix Nobel de physique 2025 a été attribué à Jean Dupont pour ses travaux sur la fusion froide. »

Le LLM a peut-être été entraîné avant 2025, ou a tout simplement comblé un vide.

4.2 Limite n°2 : Le knowledge cutoff

Définition

Knowledge cutoff : date à laquelle s'arrête le corpus d'entraînement d'un LLM. Le modèle ignore par construction tout événement postérieur.

Un modèle entraîné jusqu'en janvier 2025 ne saura pas qui a gagné l'élection américaine de novembre 2025, ne connaîtra pas la version actuelle de votre framework préféré, etc.

4.3 Limite n°3 : Pas d'accès aux données privées

Un LLM grand public n'a jamais lu votre documentation interne, vos contrats clients, votre base de connaissances métier, vos tickets Jira ou vos emails. Il ne peut donc pas répondre à : « Quel est notre processus de validation des congés ? »

4.4 Limite n°4 : Absence de sources et de traçabilité

Quand un LLM répond, il ne dit pas *d'où vient* l'information. Pour un usage professionnel (juridique, médical, audit), cette absence de traçabilité est rédhibitoire.

4.5 Limite n°5 : La fenêtre de contexte

Même avec 1M de tokens, on ne peut pas charger *toute* la documentation d'une entreprise. Et plus on remplit la fenêtre, plus le coût et la latence augmentent — et plus la qualité peut se dégrader (« lost in the middle »).

4.6 Limite n°6 : Le coût du fine-tuning

Une solution alternative serait de *réentraîner* le modèle sur ses propres données (fine-tuning). Mais :

- C'est coûteux (calcul, expertise).
- Ce n'est pas mis à jour automatiquement à chaque nouveau document.
- Cela ne donne pas non plus de sources.
- Cela peut provoquer du *catastrophic forgetting* (le modèle oublie ce qu'il savait).

4.7 Synthèse des limites

Limite	Conséquence	Le RAG aide-t-il ?
Hallucinations	Information fausse	Oui, en ancrant la réponse dans des documents
Knowledge cutoff	Pas d'actualité	Oui, en injectant du contenu à jour
Données privées	Impossible de répondre	Oui, par recherche dans la base interne
Pas de sources	Aucune traçabilité	Oui, on connaît les documents récupérés
Contexte limité	Coût/qualité	Oui, on n'envoie que les passages utiles
Coût fine-tuning	Lourd, peu réactif	Oui, le RAG est plus léger et dynamique

TABLE 2 – Comment le RAG répond aux limites des LLM.

Questions de réflexion

1. Pourquoi un LLM ne peut-il pas savoir ce qui s'est passé hier ?
2. Qu'est-ce qu'une hallucination ? Pourquoi le LLM en produit-il ?
3. Citez deux limites des LLM qui posent problème en entreprise.

5 Positionnement du RAG : Prompt Engineering vs Fine-tuning vs RAG

Avant d'entrer dans le détail du RAG, situons-le par rapport aux deux autres approches couramment utilisées pour adapter un LLM à un cas d'usage.

5.1 Prompt Engineering

Définition

Prompt Engineering : art de formuler la requête (prompt) pour obtenir la meilleure réponse possible d'un LLM, sans modifier ni le modèle ni ses données.

Avantages : rapide, gratuit, n'importe quel utilisateur peut le faire.

Limites : ne résout ni le knowledge cutoff, ni l'accès aux données privées.

5.2 Fine-tuning

Définition

Fine-tuning : réentraînement (partiel) d'un LLM sur un jeu de données spécifique pour spécialiser son comportement, son style ou sa connaissance.

Idéal pour : imiter un style, apprendre un format précis, instruire un comportement.

Mal adapté pour : injecter de nouvelles connaissances factuelles qui changent souvent.

5.3 RAG

Définition

RAG (Retrieval-Augmented Generation) : technique consistant à *recupérer* dynamiquement des documents pertinents au moment de la requête et à les fournir au LLM comme contexte avant qu'il génère la réponse.

5.4 Matrice de décision

Critère	Prompt Eng.	Fine-tuning	RAG
Coût initial	Très faible	Élevé	Moyen
Coût récurrent	Faible	Faible	Moyen (infra)
Mise à jour des connaissances	Manuelle	Réentraînement	Automatique
Sources / traçabilité	Non	Non	Oui
Adapté aux données privées	Limité	Oui	Oui
Maîtrise du style	Limitée	Excellente	Limitée
Complexité technique	Faible	Élevée	Moyenne

TABLE 3 – Comparaison des trois approches.

À retenir

Ces trois approches ne s'excluent pas. En production, on les **combine** souvent : un LLM éventuellement fine-tuné, couplé à un RAG pour la connaissance factuelle, le tout piloté par un prompt soigneusement conçu.

6 Qu'est-ce qu'un système RAG ?

6.1 Définition complète

Définition

Un **système RAG** est une architecture qui, face à une question utilisateur, (1) recherche dans une base de documents les passages les plus pertinents, (2) construit un *prompt augmenté* contenant ces passages, puis (3) demande à un LLM de produire la réponse en s'appuyant explicitement sur ce contexte.

6.2 L'analogie pédagogique : l'examen à livre ouvert

Imaginez un étudiant brillant mais qui n'a pas révisé l'examen précis du jour. Plutôt que de le forcer à répondre de mémoire (risque d'invention), on lui dit :

« Voici quelques pages du cours qui parlent justement de cette question. Réponds en t'appuyant sur ces pages, et cite-les. »

C'est exactement ce que fait un RAG. Le LLM est l'étudiant brillant ; la base documentaire est le manuel ; le moteur de recherche est sa capacité à trouver vite les bonnes pages.

6.3 Origine historique

Anecdote

Le terme « Retrieval-Augmented Generation » a été popularisé par un *papier de Facebook AI Research en 2020* : Lewis et al., « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». Les auteurs y montraient qu'un modèle couplé à un retriever surpassait des modèles purement génératifs sur les tâches nécessitant des connaissances factuelles.

6.4 Schéma général

[IMAGE À INSÉRER : schéma classique d'architecture RAG montrant : Question utilisateur → Embedding → Recherche vector store → Top-k documents → Prompt augmenté → LLM → Réponse. Chercher « RAG architecture diagram » sur Google Images]

6.5 Ce qu'un RAG n'est pas

- Ce n'est pas un moteur de recherche classique (Google-like). Le RAG *génère* une réponse en langage naturel.
- Ce n'est pas un fine-tuning. Le modèle n'est pas modifié.
- Ce n'est pas une garantie absolue contre les hallucinations : si la base est vide ou de mauvaise qualité, le LLM peut toujours inventer.

7 Fonctionnement détaillé d'un RAG

Un système RAG fonctionne en **deux phases bien distinctes** : une phase d'*indexation* (hors-ligne, préparatoire) et une phase de *retrieval + generation* (en ligne, à chaque question).

7.1 Phase 1 — L'indexation (offline)

Cette phase prépare la base de connaissances. Elle est exécutée une fois (puis à chaque ajout/-modification de documents).

[IMAGE À INSÉRER : pipeline d'indexation : Documents → Loader → Chunking → Embeddings → Vector Store. Chercher « RAG indexing pipeline » sur Google Images]

7.1.1 Étape 1.1 — Chargement des documents

On récupère les sources de connaissance dans des formats variés : PDF, Word, HTML, Markdown, base de données SQL, API, emails, transcriptions audio, etc. Chaque format requiert un *loader* dédié.

7.1.2 Étape 1.2 — Le chunking (découpage)

Définition

Chunking : opération qui consiste à découper chaque document en passages plus petits (*chunks*), généralement de quelques centaines à quelques milliers de caractères.

Pourquoi découper ?

- Pour pouvoir rechercher avec précision (on veut le bon paragraphe, pas tout le document).
- Pour respecter la fenêtre de contexte du LLM (on n'envoie que les chunks pertinents).
- Pour améliorer la qualité des embeddings (un vecteur représente mieux un paragraphe qu'un livre entier).

Stratégies courantes :

- **Fixed-size** : découpage par nombre fixe de caractères (le plus simple).
- **Recursive character splitting** : découpe en respectant la hiérarchie naturelle (paragaphes, phrases...).
- **Semantic chunking** : découpe selon les frontières sémantiques détectées par un modèle.
- **Document-aware** : respecte la structure (titres, sections) du document source.

Attention

Le **chunk overlap** (chevauchement) est un paramètre crucial : on garde quelques dizaines de tokens en commun entre chunks consécutifs pour éviter qu'une information à la frontière de deux chunks soit perdue.

7.1.3 Étape 1.3 — La vectorisation (embeddings)

Chaque chunk est passé à un **modèle d'embedding** qui le transforme en vecteur numérique de dimension fixe (par ex. 384, 768, 1536, 3072).

Exemple

"Le RAG combine recherche et génération."
→ [0.12, -0.05, 0.88, ..., 0.03] (1536 nombres réels)

Deux chunks sémantiquement proches auront des vecteurs proches (distance cosinus faible). C'est ce qui rend possible la recherche par similarité.

[IMAGE À INSÉRER : représentation visuelle de l'espace vectoriel avec des points colorés regroupés par thématique. Chercher « semantic search vector space » sur Google Images]

Quelques modèles d'embeddings courants :

- **OpenAI** : text-embedding-3-small, text-embedding-3-large
- **Cohere** : embed-v3
- **HuggingFace / open-source** : all-MiniLM-L6-v2, BGE, E5, Sentence-BERT
- **Voyage AI** : voyage-3

7.1.4 Étape 1.4 — Stockage dans un vector store

Définition

Vector store (base vectorielle) : système de stockage spécialisé permettant d'enregistrer des millions de vecteurs et d'y effectuer des recherches de similarité très rapides (algorithmes ANN — Approximate Nearest Neighbor).

Le vector store stocke pour chaque chunk : le vecteur, le texte original, et des métadonnées (source, date, auteur, page...).

7.2 Phase 2 — Retrieval + Generation (online)

Cette phase est exécutée à chaque requête utilisateur.

[IMAGE À INSÉRER : pipeline d'inférence : Question → Embedding question → Recherche top-k → Construction prompt → LLM → Réponse. Chercher « RAG query pipeline » sur Google]

7.2.1 Étape 2.1 — Vectorisation de la requête

La question de l'utilisateur est transformée en vecteur avec le **même modèle d'embedding** que celui utilisé à l'indexation. C'est impératif : les vecteurs de la question et des chunks doivent vivre dans le même espace.

7.2.2 Étape 2.2 — Recherche de similarité

Le vector store retourne les **top-k** chunks les plus proches du vecteur de la question (typiquement $k = 3$ à 10). La proximité est mesurée par :

- la **similarité cosinus** (la plus courante) ;
- la distance euclidienne ;
- le produit scalaire.

7.2.3 Étape 2.3 — Construction du prompt augmenté

On assemble un prompt structuré du type :

```
Tu es un assistant qui répond aux questions en t'appuyant uniquement sur le contexte
fourni. Si la réponse ne s'y trouve pas, dis-le.
```

```
CONTEXTE :
```

```
{chunk_1}
```

```
{chunk_2}
```

```
{chunk_3}
```

```
QUESTION : {question_utilisateur}
```

```
RÉPONSE :
```

7.2.4 Étape 2.4 — Génération par le LLM

Le LLM reçoit le prompt augmenté et génère une réponse. Idéalement, il cite les sources via les métadonnées attachées aux chunks récupérés.

À retenir

Le LLM **ne fait pas de recherche lui-même**. C'est le retriever (le vector store) qui trouve les passages. Le LLM ne fait que *rédigier* la réponse à partir du contexte fourni.

Questions de réflexion

1. Pourquoi a-t-on besoin de découper les documents en chunks ?
2. À quoi correspond la similarité cosinus dans un RAG ?
3. Pourquoi doit-on utiliser le même modèle d'embedding à l'indexation et à la requête ?

8 Les composants techniques d'un RAG

Passons en revue chaque brique technique d'un système RAG.

8.1 Document loaders

Outils qui extraient le texte (et parfois la structure) depuis des fichiers ou des APIs.

- **PDF** : PyPDF, PDFPlumber, Unstructured, PyMuPDF
- **Web** : BeautifulSoup, Scrapy, Playwright
- **Docs Office** : python-docx, openpyxl
- **APIs** : connecteurs Notion, Confluence, Slack, GitHub, Google Drive
- **Audio** : Whisper pour la transcription
- **Images / scans** : OCR (Tesseract, AWS Textract, GPT-4 Vision)

8.2 Text splitters / chunkers

Stratégie	Principe	Quand l'utiliser
Fixed-size	Découpage caractères/tokens fixes	Prototypage rapide
Recursive	Respecte paragraphes/phrases	Cas général (par défaut)
Markdown-aware	Respecte les titres MD	Documentation technique
Code-aware	Respecte fonctions/classes	Bases de code
Semantic	Frontières sémantiques (modèle)	Textes longs et continus

TABLE 4 – Stratégies de chunking.

8.3 Modèles d'embeddings

Critères de choix :

- **Qualité** : benchmarks (MTEB) sur des tâches similaires à la vôtre.
- **Langue** : certains modèles sont anglais-only, d'autres multilingues.
- **Dimension** : plus elle est élevée, plus c'est précis mais coûteux en stockage et calcul.
- **Coût** : API payante (OpenAI, Cohere) vs modèles open-source auto-hébergés (BGE, E5).
- **Latence** : l'embedding d'une question doit être quasi-instantané.

8.4 Vector stores

Vector store	Caractéristiques	Cas d'usage
Chroma	Open-source, simple, local	Prototypage, petits projets
FAISS	Bibliothèque Facebook, ultra-rapide	Recherche locale, intégration custom
Pinecone	SaaS managé, scalable	Production cloud
Weaviate	Open-source, hybride, GraphQL	Recherche hybride, métadonnées riches
Qdrant	Open-source, écrit en Rust	Production self-hosted
Milvus	Open-source, distribué	Très gros volumes
pgvector	Extension PostgreSQL	Si PG est déjà dans la stack
Elasticsearch	Recherche hybride mature	Existant Elastic

TABLE 5 – Principaux vector stores en 2026.

8.5 LLM de génération

- **API cloud** : OpenAI (GPT-4, GPT-4o), Anthropic (Claude), Google (Gemini), Mistral, Cohere.
- **Auto-hébergés** : Llama, Mistral, Qwen, Gemma via Ollama, vLLM, TGI.

8.6 Frameworks d'orchestration

- **LangChain** : leader, écosystème très riche, large communauté. *Nous l'utiliserons en TP.*
- **LlamaIndex** : spécialisé RAG, abstractions de données puissantes.
- **Haystack** : par deepset, mature en production.
- **DSPy** : approche programmatique, optimisation automatique des prompts.

9 Les types de RAG

À mesure que le domaine a mûri, plusieurs variantes et évolutions du RAG sont apparues. Comprendre cette taxonomie aide à choisir la bonne architecture selon le besoin.

9.1 Naive RAG (RAG de base)

C'est le RAG décrit dans la section précédente : indexation simple, recherche par similarité, génération directe. Il a les défauts de ses qualités :

- Sensible à la formulation de la question (mots-clés différents → mauvais retrieval).
- Pas de filtrage des chunks non pertinents.
- Pas de reformulation, pas de raisonnement multi-étapes.

9.2 Advanced RAG

Introduit des optimisations *avant* et *après* le retrieval.

9.2.1 Pré-retrieval

- **Query rewriting** : reformuler la question pour la rendre plus claire.
- **Query expansion** : générer plusieurs variantes de la question.
- **Query decomposition** : découper une question complexe en sous-questions.
- **HyDE** (*Hypothetical Document Embeddings*) : générer une réponse hypothétique, l'embedder, puis chercher des chunks similaires (souvent meilleur que d'embedder la question seule).

9.2.2 Post-retrieval

- **Re-ranking** : trier les chunks récupérés avec un modèle plus précis (cross-encoder type Cohere Rerank, BGE Reranker).
- **Filtrage** : éliminer les chunks à faible score ou sans rapport.
- **Compression contextuelle** : condenser les chunks pour ne garder que les passages utiles.

9.3 Modular RAG

Pousse le concept plus loin : le RAG devient un *graphe de modules* (retrievers, transformateurs, agents) qui s'enchaînent, se branchent, voire bouclent. C'est l'approche favorisée par DSPy ou LangGraph.

9.4 Agentic RAG

Définition

Agentic RAG : système où un *agent LLM* décide lui-même quand chercher, quoi chercher, dans quelle source, et boucle jusqu'à obtenir une réponse satisfaisante.

L'agent peut :

- interroger plusieurs sources (vector store, API, web, base SQL),
- reformuler ses recherches,

- décomposer la tâche,
- s'auto-évaluer.

9.5 Self-RAG

Le modèle est entraîné à *générer des tokens spéciaux* indiquant quand récupérer et quand répondre directement. Il apprend à juger lui-même si le contexte récupéré est pertinent et si la réponse générée est fidèle.

9.6 Corrective RAG (CRAG)

Ajoute une *boucle de correction* : un évaluateur juge la pertinence des documents récupérés. Selon le verdict (Correct, Ambigu, Incorrect), le système répond, re-cherche, ou fait appel à une recherche web externe.

9.7 GraphRAG

Définition

GraphRAG : variante où la base de connaissance n'est pas seulement un ensemble de chunks vectorisés, mais un **graphe de connaissances** (entités, relations) construit automatiquement à partir des documents.

Avantage : capture les relations multi-saut (« Quel est le PDG de la filiale française de l'entreprise rachetée en 2023 par X ? »).

9.8 HyDE

Au lieu d'embedder directement la question, on demande au LLM de générer une *réponse hypothétique*, qu'on embedde ensuite. Souvent, cette réponse hypothétique est sémantiquement plus proche des bons chunks que la question elle-même.

9.9 Multi-modal RAG

Étend le RAG aux images, tableaux, schémas, audio, vidéo. Utilise des embeddings multimodaux (CLIP, des LLM vision) pour indexer et rechercher du contenu non-textuel.

9.10 Tableau comparatif

Type	Apport clé	Complexité	Maturité
Naive RAG	Base de référence	Faible	Production
Advanced RAG	Reformulation, reranking	Moyenne	Production
Modular RAG	Architecture flexible	Moyenne/Élevée	Production
Agentic RAG	Décision autonome	Élevée	Croissante
Self-RAG	Auto-évaluation	Élevée	Recherche/Pilote
CRAG	Correction par feedback	Élevée	Croissante
GraphRAG	Relations multi-saut	Élevée	Croissante
HyDE	Meilleur retrieval	Faible	Production
Multi-modal RAG	Données non-textuelles	Moyenne/Élevée	Croissante

TABLE 6 – Vue d’ensemble des types de RAG.

À retenir

Commencez **toujours** par un Naive RAG. Mesurez ses performances. Ajoutez des techniques avancées *seulement* si elles répondent à un problème identifié, pas par effet de mode.

10 Évaluation des systèmes RAG

10.1 Pourquoi évaluer ?

Un RAG qui semble « bien marcher » en démo peut produire des réponses catastrophiques en production. L’évaluation systématique est indispensable pour :

- comparer plusieurs configurations (modèle d’embedding A vs B, chunk size 500 vs 1000...);
- détecter les régressions après une mise à jour ;
- identifier les classes de questions qui posent problème ;
- construire la confiance des utilisateurs et des décideurs.

10.2 La notion de golden dataset

Définition

Golden dataset : jeu de questions de référence, accompagné des réponses attendues et idéalement des documents qui contiennent ces réponses. Il sert de mètre étalon pour mesurer les performances du RAG.

Il peut être :

- construit manuellement par des experts métier (le plus fiable) ;
- généré automatiquement par un LLM puis validé humainement (plus rapide).

10.3 Métriques de retrieval

Évaluent uniquement la qualité de la *récupération* (avant la génération).

- **Precision@k** : parmi les k chunks récupérés, combien sont vraiment pertinents ?
- **Recall@k** : parmi tous les chunks pertinents existants, combien ont été récupérés dans le top-k ?
- **MRR** (Mean Reciprocal Rank) : à quelle position apparaît le premier chunk pertinent ? Plus c'est haut, mieux c'est.
- **NDCG** : tient compte de l'ordre et de la pertinence graduée des chunks.
- **Context Relevance** (RAGAS) : les chunks récupérés sont-ils pertinents pour la question ?

10.4 Métriques de génération

Évaluent la qualité de la *réponse* produite par le LLM.

- **Faithfulness / Groundedness** : la réponse est-elle *fidèle* au contexte récupéré (pas d'hallucination) ?
- **Answer Relevance** : la réponse répond-elle effectivement à la question ?
- **Answer Correctness** : la réponse est-elle factuellement correcte (par rapport au gold) ?
- **BLEU / ROUGE** : métriques classiques de similarité textuelle, moins utilisées en RAG car peu sémantiques.

10.5 Frameworks d'évaluation

Framework	Caractéristiques
RAGAS	Le plus populaire. Métriques RAG dédiées (faithfulness, context precision, etc.).
TruLens	Observabilité + évaluation, dashboard intégré.
DeepEval	Approche style <i>pytest</i> pour les LLM.
LangSmith	Produit LangChain : traces, datasets, évaluations.
Phoenix (Arize)	Observabilité open-source.

TABLE 7 – Frameworks d'évaluation de RAG.

10.6 La boucle d'amélioration

[IMAGE À INSÉRER : boucle Évaluer → Identifier les faiblesses → Modifier (chunking, modèles, prompt) → Réévaluer. Chercher « evaluation feedback loop » sur Google Images]

À retenir

On n'optimise que ce que l'on mesure. Sans évaluation systématique, vous « bricolez à l'aveugle » et risquez d'introduire des régressions invisibles.

Questions de réflexion

1. Qu'est-ce qu'un golden dataset ? Comment le construit-on ?
2. Quelle est la différence entre Precision@k et Recall@k ?
3. Pourquoi la métrique de *faithfulness* est-elle si importante pour un RAG ?

11 Avantages, limites et pièges du RAG

11.1 Les avantages

- **Connaissance à jour** : il suffit de réindexer pour intégrer de nouveaux documents.
- **Traçabilité** : chaque réponse peut citer ses sources, indispensable en juridique, médical, audit.
- **Données privées** : les documents ne quittent pas votre infrastructure (en self-hosting).
- **Coût maîtrisé** : bien moins cher qu'un fine-tuning complet.
- **Réduction des hallucinations** : la réponse est ancrée dans un contexte concret.
- **Flexibilité** : changer de LLM ou de vector store sans tout refaire.

11.2 Les limites

- **Qualité dépendante du retrieval** : « Garbage in, garbage out. » Si on ne trouve pas le bon chunk, le LLM ne pourra pas répondre.
- **Latence accrue** : chaque requête fait au moins 2 appels (embedding + LLM) plus la recherche.
- **Complexité opérationnelle** : pipeline d'indexation, monitoring, gestion des mises à jour.
- **Coûts d'infrastructure** : vector store, modèles d'embeddings, observabilité.
- **Limite intrinsèque** : ne capture pas bien les questions *transversales* qui nécessitent une vue d'ensemble (mieux servies par GraphRAG ou de la summarisation).

11.3 Pièges classiques

Attention

- **Chunks trop grands** : la recherche perd en précision, le contexte injecté est dilué.
- **Chunks trop petits** : information fragmentée, perte de contexte.
- **Pas de chunk overlap** : phrases coupées en plein milieu, perte d'information.
- **Mauvais modèle d'embedding** : anglais-only sur du contenu français, modèle peu performant.
- **Pas de gestion des métadonnées** : impossible de filtrer par date, auteur, type de document.
- **Prompt système trop laxiste** : le LLM répond « de mémoire » au lieu de s'en tenir au contexte.
- **Top-k trop bas** : le bon chunk est en position 6 alors qu'on récupère 5.
- **Pas d'évaluation** : on déploie en aveugle.
- **Données sales** : documents obsolètes, dupliqués, contradictoires.

12 Cas d'usage concrets

12.1 Assistants documentaires d'entreprise

Le cas d'usage le plus répandu. Un employé pose une question en langage naturel sur la politique RH, les procédures internes, la documentation technique, et reçoit une réponse sourcée.

12.2 Support client

Le RAG répond instantanément aux questions fréquentes en s'appuyant sur la base de connaissances support, les FAQ, les anciens tickets résolus. Il escalade aux humains uniquement les cas complexes.

12.3 Assistants juridiques

Recherche dans des corpus de jurisprudence, contrats, textes de loi. La traçabilité est ici absolument critique : chaque affirmation doit pouvoir être vérifiée.

12.4 Assistants médicaux

Aide au diagnostic, recherche dans la littérature médicale, accès aux protocoles internes. *Toujours sous supervision humaine*, jamais en autonomie.

12.5 Assistants techniques / DevOps

Recherche dans la documentation technique, les runbooks, les post-mortems, le code. Pratique pour onboarder les nouveaux développeurs.

12.6 Assistants éducatifs

Tuteurs personnalisés qui répondent en s'appuyant sur le matériel pédagogique d'un cours, d'un manuel, d'une bibliothèque.

12.7 Recherche académique et veille

Synthèse de littérature scientifique, exploration de corpus, génération d'états de l'art assistée.

[IMAGE À INSÉRER : illustration variée de cas d'usage en entreprise — RH, support, juridique, etc. Chercher « enterprise AI use cases illustration »]

13 Écosystème et outils

13.1 LangChain

LangChain est aujourd'hui le framework d'orchestration de référence pour construire des applications LLM, y compris les RAG. Il fournit :

- des abstractions unifiées (loaders, splitters, embeddings, vector stores, LLM) ;
- des *chaînes* prédéfinies (RetrievalQA, ConversationalRetrievalChain) ;
- LangGraph pour les workflows complexes et agentiques ;
- LangSmith pour le monitoring et l'évaluation.

Avantages : maturité, écosystème immense, communauté très active.

Inconvénients : couches d'abstraction parfois lourdes, API qui évolue vite.

13.2 Alternatives

- **LlamaIndex** : pensé spécifiquement pour le RAG, très bon sur les structures de données complexes.
- **Haystack** : par deepset, mature en production, pipelines explicites.
- **DSPy** : paradigme programmatique, optimisation automatique des prompts.
- **Semantic Kernel** : par Microsoft, intégré à l'écosystème .NET / Azure.

13.3 Providers LLM

- Cloud : OpenAI, Anthropic, Google, Cohere, Mistral, AWS Bedrock, Azure OpenAI.
- Open-weights / self-hosting : Ollama, vLLM, HuggingFace TGI, LMStudio.

13.4 Providers embeddings

- Cloud : OpenAI, Cohere, Voyage AI, Google.
- Open-source : Sentence-Transformers, BGE, E5, GTE (via HuggingFace).

14 Préparation aux séances pratiques

14.1 Ce que nous allons construire

Séance 2 (Pratique 1/2) : nous construirons un **RAG simple mais propre** de bout en bout, avec LangChain et un vector store local (Chroma ou FAISS).

- Mise en place de l'environnement Python.
- Chargement de documents (PDF ou Markdown).
- Chunking et embeddings.
- Indexation dans un vector store.
- Première chaîne RAG fonctionnelle.
- Tests sur quelques questions.

Séance 3 (Pratique 2/2) : nous itérons sur ce RAG pour le rendre *robuste*.

- Mise en place de l'évaluation avec RAGAS.
- Comparaison de plusieurs stratégies de chunking.

- Ajout de techniques avancées (re-ranking, query rewriting).
- Interface utilisateur simple (Streamlit ou Gradio).
- Discussion sur la mise en production.

14.2 Prérequis techniques

- Python 3.10+ installé.
- Un éditeur (VS Code recommandé).
- Une clé d'API OpenAI ou équivalent (ou Ollama installé localement).
- Compte GitHub pour cloner le code.

14.3 Pour aller plus loin avant la séance 2

Quelques ressources recommandées :

- Le papier fondateur : Lewis et al., 2020, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
- La documentation officielle de LangChain : <https://python.langchain.com>.
- Le *LangChain Academy* (cours gratuits).
- La chaîne YouTube *LangChain* pour des démos vidéo.
- Le blog *Hugging Face* pour les modèles d'embeddings.

Glossaire

Agent	LLM auquel on donne la capacité de planifier, de choisir et d'utiliser des outils de manière autonome.
ANN	<i>Approximate Nearest Neighbor</i> . Algorithme de recherche du plus proche voisin approximé, utilisé pour la recherche rapide dans des vector stores.
Chunk	Passage de document, unité de base indexée et récupérée par un RAG.
Embedding	Vecteur numérique représentant le sens d'un texte.
Faithfulness	Métrique mesurant si la réponse est fidèle au contexte récupéré.
Fenêtre de contexte	Nombre maximal de tokens en entrée d'un LLM.
Fine-tuning	Réentraînement spécialisé d'un LLM sur un jeu de données ciblé.
Hallucination	Génération d'information fausse présentée avec assurance.

Hybrid search

Combinaison de recherche vectorielle et de recherche par mots-clés (BM25).

Knowledge cutoff

Date butoir des données d'entraînement d'un LLM.

LLM

Large Language Model. Modèle de langage massif.

Prompt engineering

Pratique de conception de prompts efficaces.

Re-ranker

Modèle qui réordonne les chunks récupérés selon leur pertinence réelle.

Retriever

Composant qui cherche les chunks pertinents (souvent un vector store).

Token

Unité de base manipulée par le LLM (sous-mot).

Vector store

Base de données spécialisée dans la recherche par similarité vectorielle.

Bibliographie sélective

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
- Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
- Gao, Y. et al. (2023). *Retrieval-Augmented Generation for Large Language Models : A Survey*.
- Edge, D. et al. (2024). *From Local to Global : A Graph RAG Approach*. Microsoft Research.
- Asai, A. et al. (2023). *Self-RAG : Learning to Retrieve, Generate and Critique through Self-Reflection*.
- Yan, S. et al. (2024). *Corrective Retrieval Augmented Generation (CRAG)*.
- Documentation officielle LangChain : <https://python.langchain.com>
- Documentation officielle LlamaIndex : <https://docs.llamaindex.ai>
- RAGAS : <https://docs.ragas.io>
- MTEB leaderboard (benchmark d'embeddings) : <https://huggingface.co/spaces/mteb/leaderboard>

Fin du document — Bonne lecture, et à la séance 2 pour la pratique !